

TripPlanner

Отчёт по лабораторной работе по предмету «Frontend»

Многостраничное travel-приложение с json-server

Студент: Тиуков Даниил · Группа J3213 · Lab-2 (ветка dz-5)

Стек: HTML5, CSS3, Bootstrap 5, Vanilla JS, json-server, localStorage

1. Цель работы

Разработать многостраничное travel-приложение, которое работает как обычный статический сайт с переходами между HTML-файлами, но использует имитацию backend на основе `json-server`. Ключевые сущности (пользователи, направления, маршруты, заметки, избранное, идеи, этапы поездки, участники, отзывы) хранятся в `db.json` и доступны через REST. Сессия пользователя и выбранная тема хранятся в `localStorage`.

2. Структура проекта

travel-planner/	
├ index.html	– главная (hero, преимущества, направления, СТА)
├ destinations.html	– каталог направлений (фильтры, сортировка, пагинация)
├ destination-details.html	– детальная страница направления
├ dashboard.html	– личный кабинет (профиль/маршруты/заметки/избранное)
├ collaboration.html	– совместное планирование (участники/этапы/идеи)
├ login.html	– вход
├ register.html	– регистрация
├ db.json	– данные json-server
├ package.json	– npm-скрипт для запуска сервера
├ css/style.css	– единая стилистика, светлая/тёмная тема
├ js/	
│ └ main.js	– общий каркас: тема, тосты, auth-nav
│ └ api.js	– обёртка над fetch и REST-методы
│ └ data.js	– карты лейблов (city/nature, бюджет)
│ └ auth.js	– логика входа и регистрации
│ └ destinations.js	– каталог
│ └ destination-details.js	– детальная страница
│ └ dashboard.js	– кабинет
│ └ collaboration.js	– совместное планирование
└ images/	– изображения и SVG-спрайт icons.svg

3. Запуск

В `package.json` объявлен npm-скрипт. Команда поднимает локальный REST-сервер на порту 3000, который читает и пишет в `db.json` :

```
npm install
npm run server # json-server --watch db.json --port 3000
```

HTML-страницы открываются через любой статический сервер (например, WebStorm «Open in Browser» или `npx serve .`), потому что прямое открытие через `file://` не позволяет браузеру делать `fetch` к `http://localhost:3000` .

4. Хранение данных

Все доменные сущности находятся в `db.json` и доступны как REST-коллекции. Базовый URL — `http://localhost:3000` .

Коллекция	Назначение
<code>/users</code>	Учётные записи (имя, фамилия, email, пароль, предпочтение).
<code>/destinations</code>	Каталог направлений с описанием, маршрутом, советами.
<code>/reviews</code>	Отзывы пользователей по направлениям.
<code>/routes</code>	Личные маршруты пользователя.
<code>/notes</code>	Личные заметки в кабинете.
<code>/favorites</code>	Избранные направления.
<code>/participants</code>	Участники совместной поездки.
<code>/stages</code>	Этапы общего маршрута.
<code>/shared-notes</code>	Общие заметки для группы.
<code>/ideas</code>	Идеи и количество поддержавших голосов.

В `localStorage` хранятся только две вещи: ключ `tripplannerCurrentUser` (текущая сессия) и `tripplannerTheme` (выбранная тема: `light` / `dark`).

5. Слой API (`js/api.js`)

Один объект `TravelApi` , экспортируемый в `window` , скрывает `fetch` -обязку и предоставляет именованные методы по каждому ресурсу.

- Универсальные обёртки `get / post / put / patch / delete`, выбрасывающие ошибку при `HTTP >= 400`.
- `login(email, password)` — поиск пользователя по фильтру `?email=&password=`.
- `register(userData)` — проверка уникальности email и `POST /users`.
- Группы методов для каждой сущности: маршруты, заметки, избранное, участники, этапы, общие заметки, идеи, отзывы.

6. Общий каркас (`js/main.js`)

- **Тема:** переключатель `data-theme="light|dark"` на `<html>`, выбор сохраняется в `localStorage`; при первом запуске учитывается системная `prefers-color-scheme`. Inline-скрипт в `<head>` применяет тему до отрисовки, чтобы не было «мигания» цвета.
- **Тосты:** `showToast(message, type)` создаёт Bootstrap-toast в контейнере `#toastContainer`; типы — `success / error / info`.
- **Навигация:** `highlightActiveNav` подсвечивает текущий пункт по `data-page`; `syncAuthNavigation` скрывает/показывает пункты по атрибутам `data-auth-only` и `data-guest-only`.
- **Сессия:** методы `getCurrentUser / setCurrentUser / clearCurrentUser / isAuthenticated`.
- **Утилиты:** `escapeHtml` для безопасной вставки строк, `formatDate` через `Intl.DateTimeFormat('ru-RU')`.

7. Страницы сайта

7.1. Главная — `index.html`

- Единые `navbar` и `footer` со ссылками на все разделы.
- Него с заголовком, описанием, кнопками «Начать планирование» / «Войти» и тремя метриками.
- Декоративные glass-карточки: «Путешествие недели», «Поиск направлений», «Путевые заметки», «Совместная работа».
- Блок «Возможности» — четыре карточки с нумерацией.
- Блок «Популярные направления» — четыре карточки (Стамбул, Алтай, Казань, Байкал) со ссылками на детальные страницы.
- Блок «Как это работает» (СТА-карточка) и финальный СТА с кнопками регистрации/входа.
- Кнопка переключения темы (солнце/луна).

7.2. Каталог — `destinations.html`

- Карточки направлений рендерятся из `GET /destinations`.
- Фильтры: поиск по названию, тип (*Город / Природа / Смешанный*), бюджет, длительность.
- Сортировка через dropdown: по рейтингу, по бюджету, по длительности.
- Пагинация (по 6 карточек) — Bootstrap `.pagination`.
- Кнопка «Сохранить» на карточке отправляет `POST /favorites` и блокируется, если направление уже в избранном.
- Сброс всех фильтров одной кнопкой.
- Empty-state с сообщением, если по фильтрам ничего не найдено.

7.3. Детальная страница — `destination-details.html`

- Загрузка по `?id=` из query string (`GET /destinations/:id`). Если id отсутствует или направление не найдено — отдельное состояние «Ничего не найдено».
- Него-картинка, основные мета-теги (тип, бюджет, длительность, рейтинг), локация, лучший сезон, погода.
- Списки: «Чем заняться» и «Советы перед поездкой» с иконками из SVG-спрайта.
- Аккордеон «План поездки по дням» (Bootstrap `accordion`).
- Блок отзывов: рендер списка из `GET /reviews?destinationId=...` , форма отправки `POST /reviews` с автором, оценкой и текстом.
- Кнопки: «Сохранить в избранное», «Поделиться» (копирует URL в буфер обмена), «Добавить в маршрут» — модальное окно, где можно выбрать существующий маршрут или создать новый.
- Встроенная карта Google Maps по координате `mapQuery` .

7.4. Регистрация — `register.html`

- Форма: имя, фамилия, email, пароль, подтверждение пароля, чекбокс согласия.
- Клиентская валидация — обязательные поля, формат email, минимум 6 символов в пароле, совпадение паролей, отметка чекбокса. Подсветка через `is-valid / is-invalid` Bootstrap.
- При успешной валидации — `POST /users` через `TravelApi.register`. Перед записью проверяется уникальность email — повторная регистрация на тот же email вернёт ошибку и тост.
- После регистрации пользователь автоматически входит (сессия в `localStorage`), показывается модальное окно успеха и обновляется навигация.

7.5. Вход — `login.html`

- Поля: email, пароль, чекбокс «Запомнить меня».
- Валидация: обязательность, формат email, минимальная длина пароля.
- Авторизация — `GET /users?email=...&password=...`; если найдено — пользователь кладётся в сессию и происходит редирект на `dashboard.html`; если нет — отображается алерт.

7.6. Личный кабинет — `dashboard.html`

- Защищён: при отсутствии сессии редирект на `login.html`.
- Карточка профиля: имя, email, аватар (инициалы), предпочтение типа отдыха; редактирование через модальное окно (`PATCH /users/:id`).
- Вкладки Bootstrap (`nav-pills`): «Маршруты», «Заметки», «Избранное».
- Маршруты: добавление через модальное окно (`POST /routes`), удаление (`DELETE`), привязка к направлению (выбор из выпадающего списка).
- Заметки: создание, редактирование (`PATCH`) и удаление; при редактировании форма переходит в режим обновления.
- Избранное: рендер сохранённых направлений с возможностью удаления (`DELETE /favorites/:id`).
- Статистика поверх вкладок: количество маршрутов, заметок и избранных.
- Кнопка «Выйти» очищает сессию и возвращает на главную.

7.7. Совместное планирование — `collaboration.html`

- Защищён: при отсутствии сессии редирект на `login.html`.
- Участники поездки: приглашение через модальное окно (имя, email, роль, статус), `POST /participants`.
- Общий маршрут (этапы): добавление, редактирование и удаление этапа поездки (`POST/PATCH/DELETE /stages`); `empty-state`, если этапов ещё нет.
- Общие заметки: форма «Автор + Текст», список с датой и кнопкой удаления (`POST/DELETE /shared-notes`).
- Идеи и рекомендации: форма «Заголовок + Описание», карточки с бейджем «Поддержали: N» и кнопкой «Поддержать идею» (`PATCH /ideas/:id` для инкремента `votes`).

- Все формы в карточках с тенью; единая визуальная сетка через Bootstrap-grid.

8. Стили (`css/style.css`)

- CSS-переменные двух тем (`:root` и `[data-theme="dark"]`): фон, поверхности, основной/вторичный цвет, мягкие фоны для бейджей, тени, скругления.
- Шрифты Google Fonts: `Manrope` для основного текста и `Cormorant Garamond` для крупных заголовков hero / разделов.
- Карточки направлений, маршрутов, заметок и совместной работы (`.destination-card` , `.route-card` , `.note-card` , `.collab-card`) с hover-эффектом (поднятие на брх и усиленная тень).
- Бейджи категорий: `.badge-city` , `.badge-nature` , `.badge-budget` , `.badge-soft` .
- Empty-state, glass-карточки, hero-визуал, плавающие декоративные блоки на главной.
- Адаптация под `navbar` , `footer` , формы, тосты, аккордеон, бейджи.
- Класс `.idea-actions` — горизонтальный ряд «бейдж + кнопка поддержки» внутри идеи на странице совместного планирования (no-wrap, компактные паддинги).

9. Что выполнено в последней домашке

На текущей ветке `dz-5` travel-планировщик уже укомплектован и подключён к `json-server` . Конкретные доработки последнего цикла относительно прошлой версии:

- Полный переезд с чистого `localStorage` на REST-эндпоинты `json-server` (`db.json`) — все пользовательские сущности теперь читаются и пишутся через `fetch` , а `localStorage` отвечает только за сессию и тему.
- Регистрация пользователей делает реальный `POST /users` с проверкой уникальности email и автоматическим входом после успеха.
- Вход выполняется как `GET /users?email=...&password=...` ; неверные данные отображаются в виде Bootstrap-алерта.
- Правки UI: горизонтальная компоновка кнопок «Поддержать идею» в карточках идей (`.idea-actions` в `style.css` + класс в `collaboration.js`), чтобы кнопка не переносила текст и помещалась в карточку на узких колонках.
- Поддержка тёмной темы через `data-theme` + inline-скрипт в `<head>` , чтобы избежать «мигания» цвета при загрузке страниц.

10. Bootstrap-компоненты

В проекте используются: `navbar` `cards` `forms` `buttons` `modal` `badges` `alerts` `tabs / pills` `dropdown` `pagination` `toast` `accordion`

11. Сценарии проверки

1. **Регистрация:** открыть `register.html`, заполнить корректные поля → попадаем в кабинет. Повторная отправка с тем же email — должна показать тост-ошибку.
2. **Вход:** `login.html` с email/паролем созданного пользователя → редирект в кабинет; неверные данные — алерт.
3. **Каталог:** применить фильтры, сортировку, пагинацию; сохранить направление в избранное и убедиться, что оно появилось в кабинете.
4. **Деталь направления:** открыть карточку, оставить отзыв, добавить в маршрут (новый/существующий).
5. **Кабинет:** добавить и удалить маршрут, создать/отредактировать/удалить заметку, обновить профиль через модальное окно, выйти из аккаунта.
6. **Совместное планирование:** пригласить участника, добавить и отредактировать этап, оставить общую заметку, добавить идею и поддержать её.
7. **Тема:** переключить тему — выбор сохраняется и применяется на других страницах без мигания.
8. **Защита разделов:** разлогиниться и попробовать открыть `dashboard.html` или `collaboration.html` напрямую — редирект на страницу входа.

12. Итог

Получено многостраничное travel-приложение, которое:

- работает на стеке HTML + CSS + Bootstrap + Vanilla JS;
- имитирует backend через `json-server` (CRUD по всем сущностям);
- поддерживает регистрацию/вход и защищённые разделы;
- имеет светлую и тёмную темы и единый визуальный язык;
- покрывает реальные сценарии планирования поездки — от поиска направления и сохранения маршрута до совместной работы команды.